

Common Ground / Leeds — Supplementary Technical Report

Project URL: <https://opalus12138.github.io/chen/> **Module:** GEOG5870 Web-Based GIS **Word count:** under 1,000 words, including headings and references

1. What the project is for

Common Ground / Leeds is an exploratory map of recorded public green spaces in Leeds. I chose this topic because a list of sites says little about how green space is distributed across a city. The map lets visitors see the overall pattern, search for a named place, compare types of space and find the sites with the largest recorded areas. It is aimed at general visitors rather than GIS specialists.

The map is not an access or health map. A polygon shows that a place has been recorded, but not whether it has a convenient entrance, is well maintained, or can be reached quickly on foot. The website states this limit rather than making a claim that the data cannot support.

2. Requirements and use cases

The assessment brief asks for a functioning website built mainly with HTML, CSS, JavaScript and Leaflet, supported by explanatory material and data citations. I translated this into a simple user journey: a visitor should understand the question behind the map, view Leeds at city scale, find a particular green space, filter by type and inspect one site without opening separate GIS software.

The page includes an introduction, a searchable Leaflet map, a category selector, popups, a reset button and live summary figures. It also has a category chart and a ranked list of the five largest recorded spaces. These parts are interactive. Clicking a category filters the map; clicking a ranked space moves the map to it, highlights the feature and opens its popup. The Types and Area buttons offer two readings of the same layer. The first keeps the map consistent, while the second uses light and dark greens to show area bands.

I also included data credits, a limitations section, keyboard focus styles, responsive layout and reduced-motion handling. If the data request fails, the map displays a short error message instead of a blank panel.

3. Data and processing

The data comes from Leeds City Council's Public Green Spaces ArcGIS REST feature service. The service returns a maximum of 1,000 records per request, so I used two paginated requests to retrieve all 1,781 records. I requested the geometry in WGS84 for direct use in Leaflet.

The local GeoJSON keeps the fields needed by the site: site name, source ID, ward, recorded area, ownership and maintenance information. The original typology values are abbreviated codes. I checked the service metadata before translating them into labels such as "Local park", "Allotment" and "Natural and semi-natural". I did not infer a category from a site name. Missing optional values are shown as "Not available".

The browser loads the local file once. JavaScript keeps the full feature list and creates a filtered list from the selected category and search text. After a filter changes, Leaflet redraws the visible features and the summary values are recalculated from that same list. This keeps the figures beside the map tied to what the visitor is viewing.

The Area view uses three bands: under 5 hectares, 5–20 hectares and over 20 hectares. A pale colour is used when an area value is missing. The five largest polygon features receive numbered markers and appear in the ranked list. Both use the same source IDs.

4. Technology and design choices

I used semantic HTML for headings, labels, lists and page landmarks. The CSS is closer to an editorial map spread than a standard dashboard: warm paper tones, dark green, a small amount of amber and large serif headings. This gives the map room to lead the page while keeping the controls usable on a small screen.

Vanilla JavaScript keeps the assessed programming visible and avoids making a charting library the main feature. Leaflet handles the basemap, GeoJSON rendering, popups, map bounds and controls. GitHub Pages hosts the site as static files, which is enough for this prototype and avoids a database, login system and server-side credentials.

Popup values are HTML-escaped before insertion. The site does not collect visitor information, and no API keys are stored in the repository. The basemap and Leaflet are loaded over HTTPS with attribution.

5. Problems, limits and next steps

The record limit on the council service was the main implementation problem; pagination solved it. The abbreviated typology codes needed a second pass through the service metadata. The city-wide view is visually dense because many polygons overlap at a small scale. Filtering, transparency, area emphasis and the ranked list help reduce that density, although no zoom level can make every boundary equally clear.

The data describes recorded spaces, not current ground conditions. It does not provide entrances, walking times, slope, maintenance, perceived safety, user numbers or a guarantee of unrestricted public access. Recorded area is a source attribute, not an independent measure of ecological value or accessibility. The service metadata does not give a separate layer licence, so the council's open-data terms should be checked before wider redistribution.

A later version could calculate five-, ten- and fifteen-minute walking catchments, compare provision with population data, record entrances and facilities, and show when each source record was updated. An accessible table download would also make the information useful beyond the map.

References

Leeds City Council (n.d.) *Public Green Spaces*. ArcGIS REST Services. Available at: <https://mapservices.leeds.gov.uk/arcgis> and <https://www.leeds.gov.uk/opendata/terms-and-licences> (Accessed: 24 August 2026).

OpenStreetMap contributors (2026) *OpenStreetMap*. Available at: <https://www.openstreetmap.org/copyright> (Accessed: 24 August 2026).

Leaflet (2025) *Leaflet: an open-source JavaScript library for mobile-friendly interactive maps*. Available at: <https://leafletjs.com/> (Accessed: 24 August 2026).

GenAI acknowledgement

I acknowledge the use of Claude Code 2.1.241, using GPT-5.6-sol (Anthropic, <https://claude.ai/code/>), to assist with requirements interpretation, code drafting, visual refinement and technical-report editing. I checked the data, links, implementation and final files before submission.